AI Recommendation System

RAG, Embedding-Based Retrieval, Vector Search and Agent Orchestration — Explained in Simple Terms

> **The project statement**
>
> "Improved recommendation accuracy 35%, as measured by A/B telemetry, by engineering RAG and embedding-based retrieval pipelines with vector search and agent-based orchestration."

What this document covers: a plain-language summary, every component in detail, the end-to-end workflow, and a fully explained architecture diagram.

Contents

1. Overall Summary in Simple Terms

The project in plain words, an analogy, and what 35% means

2. Key Terms in Plain English

Glossary of every technical word used

3. Every Component Explained

All 26 components across five layers, with inside steps and fine details

4. End-to-End Workflow

Offline indexing, live request, A/B measurement, feedback loop

5. Architecture Diagram

The diagram plus every box and every arrow explained

6. Why Accuracy Improved by 35%

Which part fixed which weakness

7. Typical Technology Choices

Common tools for each component

8. Explaining It in an Interview

30-second pitch and likely follow-up questions

1. Overall Summary in Simple Terms

Imagine an online shop with millions of products. Every time a customer visits, the shop has to answer one question: **"Which few products should we show this person right now?"** Showing the wrong things wastes the customer's time. Showing the right things makes them happy and makes them buy more. That decision is called a **recommendation**.

The old system picked products mostly by matching keywords and looking at what was popular. It did not really "understand" what customers meant. This project replaced it with a smarter system built from three ideas:

1. **Embedding-based retrieval with vector search** — Every product and every customer request is turned into a list of numbers (a "vector") that captures its **meaning**. Things with similar meaning get similar numbers. So a search for "comfy shoes for long walks" can find "cushioned walking sneakers" even though the words are different. A special database (a vector database) finds the closest matches in milliseconds.
2. **RAG (Retrieval-Augmented Generation)** — A large language model (LLM, like the AI behind chat assistants) is very good with language but does not know the shop's real, current products. RAG fixes that: first we **retrieve** real product facts from our database, then we hand those facts to the LLM so it can **generate** a well-reasoned, accurate recommendation. The LLM works from real data instead of guessing.
3. **Agent-based orchestration** — Instead of one big program doing everything, the work is split among small specialist AI "agents" (one understands the request, one knows the user, one searches, one ranks). A "manager" agent, the orchestrator, plans the steps and coordinates them, like a project lead running a team.

To prove the new system was really better, it was tested with an **A/B test**: some customers (Group A) kept seeing the old recommendations, others (Group B) saw the new ones, and every view and click was recorded (**telemetry**). Group B's recommendations were judged correct/relevant **35% more often** than Group A's. Because both groups were real users at the same time, the improvement can be credited to the new system and not to luck or season.

> **The whole project in one analogy**
>
> Think of a very good bookshop. The **librarian** (vector search) knows where every book is and can find books by topic, not just title. The **shop assistant** (the LLM with RAG) listens to you, takes the books the librarian found, and explains which ones suit you best and why. The **store manager** (the orchestrator agent) makes sure the right staff help you in the right order. The **owner** secretly compares this new team with the old one on different customers (the A/B test) and finds that customers are happy with the suggestions 35% more often.

1.1 The five-line version

1. Collect product and user data.
2. Clean it, cut it into pieces, and convert each piece into a meaning-vector; store them in a vector database.
3. When a user asks or browses, agents understand the request, find the closest matching products by meaning, and rank them for that user.
4. An LLM writes the final recommendation using only the retrieved, real product facts, and safety checks validate it.
5. An A/B test compares old vs. new on real traffic; the new system was 35% more accurate, and the results keep feeding back to improve it.

1.2 What "35% improvement" means

It is a **relative** improvement. For example, if the old system's recommendations were relevant (clicked, bought, or rated good) 20 times out of every 100, the new system achieved 27 out of 100. The jump from 20 to 27 is 7 points, and 7 ÷ 20 = 0.35, i.e. **35% better**. Section 4.3 walks through how this is measured step by step.

> **A note on specifics**
> The project statement does not name specific tools or vendors. This document describes the standard, industry-typical way such a system is designed. Tool names (for example FAISS, Pinecone, or LangGraph) are given as common examples, and numbers in worked examples are illustrative.

2. Key Terms in Plain English

Read this table once; every later section uses these words.

Term	Simple meaning
Recommendation	A suggestion of items (products, articles, videos) that a person is likely to want.
LLM (Large Language Model)	An AI trained on huge amounts of text that can read and write language, e.g. the models behind chat assistants.
RAG (Retrieval-Augmented Generation)	"Look it up, then answer." First retrieve real facts from your own data, then let the LLM generate an answer using those facts.
Embedding	Turning text (or an image, or a user) into a list of numbers that represents its meaning. Similar meaning → similar numbers.
Vector	The list of numbers produced by an embedding, e.g. 768 numbers. Think of it as GPS coordinates on a "map of meaning".
Vector search	Finding the vectors closest to a query vector, i.e. the items whose meaning is closest to the request.
Cosine similarity	A score from -1 to 1 for how much two vectors point the same way. Close to 1 = very similar meaning.
ANN (Approximate Nearest Neighbour)	A shortcut that finds "almost certainly the closest" items very fast, instead of checking all millions one by one.
HNSW	A popular ANN method that links vectors into a multi-level "social network" so search can hop quickly towards the closest ones.
Chunk	A small piece of a longer text (e.g. one paragraph of a product description), so each piece has one clear meaning.
BM25 / keyword search	Classic search that scores documents by matching exact words. Good for model numbers, brand names, sizes.
Hybrid search	Using vector search and keyword search together and merging the results to get the best of both.
Re-ranking	A second, more careful scoring pass over the top candidates to put the very best ones first.
Agent	An AI component that has a goal, can make decisions, and can use tools (search, databases, APIs) to reach that goal.

Term	Simple meaning
Orchestration	Coordinating several agents/steps: deciding what runs, in which order, and combining results.
Prompt	The instructions and context text sent to the LLM.
Hallucination	When an LLM confidently states something that is not true. RAG and guardrails reduce this.
Guardrails	Automatic checks that block wrong, unsafe or out-of-stock suggestions before users see them.
Cache	A short-term memory of recent answers, so repeated requests are answered instantly.
A/B test	A fair experiment: users are randomly split into Group A (old) and Group B (new), and results are compared.
Telemetry	Automatic recording of what happens in the system and what users do (views, clicks, purchases, timings).
CTR (Click-Through Rate)	Clicks ÷ times shown. If 100 people see a recommendation and 5 click, CTR = 5%.
Precision@K / Hit rate	Out of the top K items shown, how many were relevant. A common way to define “recommendation accuracy”.
Statistical significance	Mathematical evidence that a difference between A and B is real and not random luck (often p-value < 0.05).
Latency	How long the system takes to respond, usually measured in milliseconds (ms).

3. Every Component Explained

The system is organised into five layers. Each component has an ID (A1, B2, D5…) that matches the boxes on the architecture diagram in Section 5.

Layer	Job in one sentence	Components
A. Data Sources	Where all raw information comes from.	A1 Product Catalog, A2 Product Content, A3 User Activity Logs, A4 User Profiles
B. Offline Indexing Pipeline	Prepares the data in advance, on a schedule, so live requests are fast.	B1 Ingestion, B2 Cleaning, B3 Chunking, B4 Embedding Model
C. Knowledge Storage	Keeps the prepared data in forms that are fast to search.	C1 Vector DB, C2 Metadata Store, C3 Keyword Index, C4 User Feature Store
D. Online Serving	Answers each live user request in well under a second.	D0 User/App, D1 API Gateway, D2 Orchestrator, D3–D6 Specialist Agents, D7 LLM Generator, D8 Guardrails, D9 Cache
E. Experimentation & Telemetry	Measures whether the new system is better (the 35%).	E1 A/B Assignment, E2 Event Tracking, E3 Metrics Pipeline, E4 Dashboard & Stats

Layer A — Data Sources

These are the “raw ingredients”. Nothing smart happens here yet; the quality of everything later depends on the quality of this data.

ID / Source	What it contains	Why the system needs it
A1 Product Catalog	Structured facts for each item: ID, name, brand, category, price, size, colour, stock level.	Gives hard facts used for filtering (in stock, under budget) and for the final answer.
A2 Product Content	Free text: descriptions, specifications, customer reviews, Q&A, FAQs, sometimes images.	This is where the “meaning” lives. It becomes the embeddings that vector search uses.
A3 User Activity Logs	Every view, search, click, add-to-cart, purchase, return, with time stamps.	Shows what people actually like; used to personalise and to learn what “relevant” means.
A4 User Profiles	Saved preferences, sizes, location, past orders, loyalty tier (with consent).	Lets the system tailor results to the person, not just the query.

Fine details

- Data arrives in two speeds: **batch** (e.g. the full catalog exported nightly) and **streaming** (e.g. clicks flowing in second by second through a message queue such as Kafka).
- Personal data is handled under privacy rules: only consented data is used, and identifiers are pseudonymised (replaced with random IDs).
- Each record keeps a **source ID** and **last-updated time** so the pipeline can process only what changed.

Layer B — Offline Indexing Pipeline

"Offline" means it runs in the background on a schedule (for example every hour, plus instantly for urgent updates like price changes). Its job is to do the heavy work in advance so that live requests stay fast.

B1 Data Ingestion (Connectors / ETL)

What it is	Programs that pull data from every source (A1–A4) into one place. ETL = Extract, Transform, Load.
Simple analogy	Delivery trucks that collect ingredients from different farms and bring them to one kitchen.
Input → Output	Raw data from databases, files, APIs and event streams → one unified raw data set.
Why it matters	Without it, data stays scattered and out of date; recommendations would be based on stale facts.

What happens inside, step by step

1. A scheduler (e.g. Airflow) starts the job at the planned time, or an event (e.g. "product updated") triggers it instantly.
2. Connectors read from each source: SQL databases, cloud storage files, product APIs, and the click-stream queue.
3. Change detection compares "last-updated" times so only new or changed records are processed (incremental load).
4. Each record is converted to one common format (e.g. JSON with standard field names).
5. Records are written to a staging area, and the job logs how many were read, changed, or failed.

Fine details

- Failed records are sent to a "dead-letter" list for review instead of stopping the whole job.
- Retries with back-off handle temporary network errors.

B2 Cleaning & Normalization

What it is	Fixes messy data so every record is complete, consistent and trustworthy.
Simple analogy	Washing, peeling and sorting the ingredients before cooking.
Input → Output	Raw unified records → clean, standardised records (also written to the Metadata Store C2).
Why it matters	Garbage in, garbage out. Dirty text creates poor embeddings and wrong recommendations.

What happens inside, step by step

1. Remove duplicates (the same product listed twice).
2. Strip noise: HTML tags, emojis where unhelpful, boilerplate like "Free shipping!" repeated on every item.
3. Standardise values: "Blk", "black", "BLACK" → "black"; sizes and currencies into one unit.
4. Fill or flag missing fields (e.g. no description → build one from the title and attributes).
5. Filter out unwanted content: spam reviews, offensive text, discontinued items.
6. Mask personal information (names, emails, phone numbers) inside reviews.
7. Send the clean structured fields (price, stock, category…) to the Metadata Store (C2).

Fine details

- Language detection allows translating or tagging non-English text.
- Data-quality checks (e.g. "price must be > 0") stop bad batches from reaching production.

B3 Chunking

What it is	Splits long text into smaller, meaningful pieces called chunks.
Simple analogy	Cutting a long newspaper into separate articles so each one is about one topic.
Input → Output	Clean long text (descriptions, reviews, FAQs) → many short chunks, each with metadata (product ID, source, position).

Why it matters	An embedding of a whole long page blurs many topics together. Small focused chunks are matched much more precisely.

What happens inside, step by step

1. Choose a chunk size, typically 200–500 tokens (roughly 150–400 words).
2. Split at natural boundaries: headings, paragraphs, sentences — never in the middle of a sentence.
3. Add an overlap of about 10–20% between neighbouring chunks so no idea is cut in half.
4. Attach metadata to each chunk: product ID, chunk type (spec / review / FAQ), language, date.
5. Optionally prepend a short title, e.g. “TrailRunner 2 — Review:”, so the chunk keeps its context.
6. Send the chunk text to the Keyword Index (C3) and to the Embedding Model (B4).

Fine details

- Short structured items (e.g. a product title + attributes) are usually kept as one chunk.
- Chunk size is tuned by experiment: too small loses context, too large loses precision.

B4 Embedding Model

What it is	An AI model that turns each chunk (and later each user query) into a vector — a list of numbers representing meaning.
Simple analogy	Giving every item GPS coordinates on a “map of meaning”. Similar items end up as neighbours on the map.
Input → Output	Text chunks → vectors (e.g. 768 or 1,536 numbers each), saved to the Vector Database (C1).
Why it matters	This is what lets the system match by meaning, not just exact words — a big part of the accuracy gain.

What happens inside, step by step

1. Chunks are grouped into batches (e.g. 256 at a time) for speed, often on GPUs.
2. Each chunk is tokenised (split into word pieces) and passed through the model.
3. The model outputs one vector per chunk.
4. Vectors are normalised (scaled to length 1) so cosine similarity can be computed quickly.
5. Each vector is stored with its chunk ID, product ID and metadata in the Vector Database (C1).

Fine details

- The **same model** must embed both the products (offline) and the user queries (online); otherwise the "maps" would not line up.
- The model can be a general one (e.g. OpenAI text-embedding, Sentence-Transformers) or **fine-tuned** on the shop's own click data so that "similar" means "bought together", which usually adds accuracy.
- When the model is upgraded, all items are re-embedded and the index is rebuilt side-by-side, then switched over with no downtime.
- Users can also get an embedding (an average of items they liked), enabling "people like you also liked" matches.

Layer C — Knowledge Storage

Prepared data is stored in four specialised stores. Each one is built for a different kind of question.

C1 Vector Database

What it is	A database designed to store millions of vectors and quickly find the ones closest to a query vector.
Simple analogy	A librarian who can instantly point to the 50 books most similar in topic to the one you describe.
Input → Output	Query vector → top-K nearest chunks with similarity scores (e.g. top 100 in ~20 ms).
Why it matters	Checking millions of items one by one would take seconds; the vector DB does it in milliseconds.

What happens inside, step by step

1. Vectors are organised into an ANN index, commonly **HNSW**: a layered graph where each vector links to its nearest neighbours.
2. A search starts at the top (sparse) layer and greedily hops closer to the query, dropping to denser layers until it reaches the closest neighbours.
3. Metadata filters (e.g. category = shoes, in_stock = true) are applied during or right after the search.
4. Results return as (chunk ID, product ID, similarity score).

Fine details

- Common choices: Pinecone, Milvus, Weaviate, Qdrant, pgvector (PostgreSQL), FAISS (library).
- Tuning knobs (like HNSW's "ef" and "M") trade a little accuracy for a lot of speed.

- Replicated across servers for reliability; sharded (split) when the data is very large.

C2 Metadata Store

What it is	A fast regular database with structured facts: price, stock, category, brand, rating, ship time.
Simple analogy	The shop's price-and-stock register.
Input → Output	Product IDs → current facts about those products.
Why it matters	Vectors know meaning, not facts. This store ensures we never recommend something out of stock or over budget.

What happens inside, step by step

1. Updated by B2 and by real-time events (e.g. stock changes).
2. Looked up by the Re-ranking Agent (D6) and Guardrails (D8) using product IDs.

Fine details

- Often a key-value store or SQL database (e.g. Redis, DynamoDB, PostgreSQL).

C3 Keyword Index (BM25)

What it is	A traditional search index that matches exact words and scores them with the BM25 formula.
Simple analogy	The index at the back of a book: look up a word, get the pages.
Input → Output	Query words → top-K chunks that contain those words, with keyword scores.
Why it matters	Vector search can miss exact things like model numbers ("XR-500"), brand names or sizes. Keyword search catches them.

What happens inside, step by step

1. Each chunk's words are stored in an inverted index (word → list of chunks).
2. BM25 rewards chunks where the query words appear often, and where those words are rare elsewhere.

Fine details

- Common choices: Elasticsearch, OpenSearch, Solr.

C4 User Feature Store

What it is	A store of ready-to-use facts about each user: preferences, size, recent views, favourite brands, user embedding.
Simple analogy	A regular customer's loyalty card that the assistant glances at before helping.
Input → Output	User ID → that user's profile features (in a few milliseconds).
Why it matters	Personalisation: two people typing the same words often want different things.

What happens inside, step by step

1. Long-term features (from A4 and history) are computed in batch.
2. Short-term features (last 30 minutes of clicks) are updated from the event stream.

Fine details

- Examples: Feast, Redis, or an in-house feature store.

Layer D — Online Serving

This layer runs live, for every request, typically finishing in **under one second**. It is where the agents work.

D0 User / App

What it is	The website or mobile app where the customer browses, searches, or chats.
Simple analogy	The customer walking up to the counter.
Input → Output	User action (search text, page view, chat message) → request to the API; later displays the recommendations.
Why it matters	It is both where requests start and where telemetry events (views, clicks) are produced.

Fine details

- Each request carries a user ID (or anonymous session ID), the query/context, device and page.
- The app logs "impression" (shown), "click" and "purchase" events to E2.

D1 API Gateway

What it is	The single front door for all requests.
Simple analogy	The receptionist who checks your ID and directs you to the right desk.
Input → Output	Incoming request → verified request forwarded to the Orchestrator, tagged with the A/B group.
Why it matters	Security, fairness and control in one place.

What happens inside, step by step

1. Checks authentication (is this a valid app/user token?).
2. Applies rate limits (stops abuse or overload).
3. Asks the A/B Assignment service (E1) which group the user belongs to and attaches it.
4. Routes Group A to the old recommender and Group B to the new agent-based pipeline.

D9 Cache

What it is	A short-term memory of recent results.
Simple analogy	The assistant remembering the answer they gave five minutes ago to the same question.
Input → Output	Request key (query + user segment) → stored answer if fresh, otherwise “miss”.
Why it matters	Saves time and LLM cost for popular, repeated requests.

Fine details

- Entries expire after a set time (e.g. 10–60 minutes) so prices and stock stay fresh.
- “Semantic caching” can reuse an answer when a new query is very close in meaning to a cached one.

D2 Agent Orchestrator (the “manager” agent)

What it is	The coordinator that plans which specialist agents to call, in what order, and combines their results.
Simple analogy	A project manager who hands tasks to specialists and assembles the final report.
Input → Output	Verified request → a plan, calls to D3–D6, then a packaged context for the LLM (D7).

Why it matters	Different requests need different steps. Splitting the work makes each part simpler, testable and better at its job.

What happens inside, step by step

1. Checks the cache (D9). If a fresh answer exists, returns it immediately.
2. Creates a plan, e.g. “1) understand query, 2) get user context, 3) retrieve, 4) re-rank, 5) generate”.
3. Runs independent steps **in parallel** (query understanding and user context at the same time) to save time.
4. Passes outputs from one agent into the next (shared “state”).
5. Checks quality: if retrieval returns too few good results, it can loosen filters or rewrite the query and retry.
6. Applies time limits to each step; if one is slow, it uses a fallback (e.g. popular items in that category).
7. Bundles the final candidates and context and sends them to the LLM Generator (D7).

Fine details

- Common frameworks: LangGraph, LangChain, LlamaIndex, Semantic Kernel, or custom code.
- Every step is traced (inputs, outputs, timings) so engineers can debug and improve.

D3 Query Understanding Agent

What it is	Works out what the user actually wants from their words or behaviour.
Simple analogy	An assistant who hears “something for my mum’s birthday, she gardens” and writes “gift, gardening, for a woman”.
Input → Output	Raw query/context → structured intent: category, filters, rewritten query, query vector.
Why it matters	Clear intent leads to much better retrieval; vague queries are the main cause of poor results.

What happens inside, step by step

1. Fixes typos and expands abbreviations (“runnin shoos” → “running shoes”).
2. Extracts hard filters: budget (“under $100”), size, colour, brand.
3. Detects intent: buy, compare, browse, or a question.
4. Rewrites the query into one or more clear search phrases (query expansion).

5. Embeds the rewritten query with the same embedding model as B4 to produce the query vector.

D4 User Context Agent

What it is	Fetches and summarises who the user is and what they have recently done.
Simple analogy	The assistant checking the loyalty card: “wears size 9, likes brand X, just looked at trail shoes”.
Input → Output	User ID → user features and a short profile summary (from C4).
Why it matters	Personalisation lifts accuracy: results fit the person, not just the words.

Fine details

- New users with no history (“cold start”) get context from their session and from popular items for similar users.
- Only privacy-approved features are used.

D5 Retrieval Agent (Hybrid Search)

What it is	Finds the best candidate products by searching both by meaning (vector) and by exact words (keyword).
Simple analogy	Two librarians: one finds books on the right topic, the other finds books with the exact title words; you combine their piles.
Input → Output	Query vector + keywords + filters → ~100 candidate products with scores.
Why it matters	This is the “R” in RAG. If the right item is not retrieved here, nothing later can recommend it.

What happens inside, step by step

1. Sends the query vector to the Vector Database (C1) for the top ~100 closest chunks.
2. Sends the query words to the Keyword Index (C3) for the top ~100 matching chunks.
3. Applies metadata filters (in stock, price range, category).
4. Merges both lists, commonly with **Reciprocal Rank Fusion (RRF)**: items ranked high in either list rise to the top.
5. Groups chunks by product, so each product appears once with its best evidence (e.g. the review that matched).

Fine details

- Can also add "collaborative" candidates: items similar to what the user recently liked (using the user embedding).

D6 Re-ranking Agent

What it is	Carefully re-scores the candidates and puts the best ones first for this specific user.
Simple analogy	After the librarians bring 100 books, an expert reads the back covers and picks the best 10 for you.
Input → Output	~100 candidates + user context → top ~10–20 ranked products.
Why it matters	First-pass search is fast but rough. Re-ranking is slower but much more precise, so it runs only on the shortlist.

What happens inside, step by step

1. Uses a **cross-encoder** model that reads the query and each candidate together and scores how well they match.
2. Combines that with business and personal signals: user preferences (C4), rating, price fit, margin, freshness (C2).
3. Adds **diversity** so the list is not 10 nearly identical items.
4. Removes anything already purchased or out of stock.
5. Outputs the final ordered shortlist.

D7 LLM Generator (the "G" in RAG)

What it is	The large language model that produces the final recommendation using only the retrieved facts.
Simple analogy	The friendly shop assistant who explains, "I'd pick these three; this one is best for flat feet because…".
Input → Output	A prompt with instructions + user context + top products and their facts → final ranked list with short reasons (structured JSON).
Why it matters	It reasons over the evidence (reviews, specs, user needs) to choose and explain, which simple scoring cannot do.

What happens inside, step by step

1. Builds a prompt: role and rules ("only recommend from the list below"), the user's need, the user context, and each candidate's facts.
2. The LLM selects and orders the best items and writes a one-line reason for each.
3. Output is requested in a strict format (e.g. JSON with product IDs and reasons) so software can read it reliably.

Fine details

- Temperature (randomness) is set low for consistent answers.
- Product IDs in the output must come from the retrieved list — this is what keeps the answer grounded.

D8 Guardrails & Validation

What it is	Automatic final checks before anything reaches the user.
Simple analogy	A supervisor who double-checks the assistant's list before handing it to the customer.
Input → Output	LLM output → approved recommendations (or a safe fallback).
Why it matters	Prevents hallucinated products, wrong prices, unsafe or inappropriate content.

What happens inside, step by step

1. Verifies every product ID exists and was in the retrieved set (no invented items).
2. Re-checks live price and stock in C2.
3. Checks the format is valid and the reasons match real product facts.
4. Applies content and policy filters (age-restricted items, offensive text).
5. If checks fail, falls back to the re-ranker's list without LLM text.
6. Stores the approved result in the cache (D9) and returns it to the app (D0).

Layer E — Experimentation & Telemetry

This layer is how the team **proved** the 35% improvement. It records what happens and compares the old and new systems fairly.

E1 A/B Assignment Service

What it is	Randomly and permanently places each user into Group A (control: old system) or Group B (treatment: new system).

Simple analogy	Flipping a fair coin for each customer, and remembering the result.
Input → Output	User ID → group label (A or B).
Why it matters	Random groups make the comparison fair: the only systematic difference is the recommender.

What happens inside, step by step

1. Hashes the user ID with the experiment name to get a stable number (so the same user always lands in the same group).
2. Splits traffic, e.g. 50/50, or starts small (5%) and ramps up when safe.
3. Records the assignment so every later event can be tagged with the group.

E2 Event Tracking

What it is	Captures user actions and system events as they happen.
Simple analogy	A tally counter for every time a recommendation is seen, clicked or bought.
Input → Output	App and server events → a clean event stream stored in a data warehouse.
Why it matters	Accurate counting is the foundation of the 35% claim.

Fine details

- Events: impression (shown), click, add-to-cart, purchase, rating, dwell time, plus latency and errors.
- Each event includes user/session ID, A/B group, recommendation ID, position on screen and time.
- Bot traffic and duplicate events are filtered out.

E3 Metrics Pipeline

What it is	Turns raw events into the numbers that define “accuracy” and health.
Simple analogy	The accountant who turns receipts into a profit report.
Input → Output	Events → daily metrics per group: precision@K/hit rate, CTR, conversion, revenue per user, latency.
Why it matters	Converts behaviour into comparable scores for A and B.

Fine details

- Primary metric (“recommendation accuracy”): e.g. share of recommendation lists where the user engaged with at least one item, or precision@10.
- Guardrail metrics: latency, error rate, cost per request, returns — the new system must not make these worse.
- Metrics are also fed back into training data (the feedback loop).

E4 Dashboard & Statistical Tests

What it is	Shows the results and tests whether the difference is real.
Simple analogy	The referee who decides whether the win is clear or just luck.
Input → Output	Metrics per group → lift %, confidence interval, p-value, ship / no-ship decision.
Why it matters	Turns “it looks better” into “it is proven better”.

Fine details

- Lift = (B − A) ÷ A. Here: +35%.
- Significance test (e.g. two-proportion z-test or bootstrap) confirms the result is very unlikely to be chance.
- Run long enough (often 2–4 weeks) to cover weekly patterns and avoid the “novelty” effect.

4. End-to-End Workflow

There are four workflows that work together: (1) preparing the knowledge offline, (2) answering a live request, (3) measuring with the A/B test, and (4) learning from results. Each step lists the component IDs involved.

4.1 Workflow 1 — Offline Indexing (preparing the library)

Runs on a schedule (e.g. hourly) and on urgent change events.

Step	What happens	In simple terms	Components
1	Scheduler starts the ingestion job.	The alarm clock rings: time to refresh the data.	B1
2	Connectors extract new/changed records from catalog, content, logs and profiles.	Trucks collect only the fresh ingredients.	A1–A4 → B1
3	Records converted into one common format.	Everything gets the same label layout.	B1
4	Duplicates removed, text cleaned, values standardised, personal info masked.	Wash and sort the ingredients.	B2
5	Structured facts (price, stock, category) saved.	Update the price-and-stock register.	B2 → C2
6	User features computed and saved.	Update each customer's loyalty card.	B1 → C4
7	Long text split into overlapping chunks with metadata.	Cut long pages into bite-sized topics.	B3
8	Chunks added to the keyword index.	Update the book's back-of-index.	B3 → C3
9	Chunks converted to vectors in batches.	Give each piece its "meaning" coordinates.	B4
10	Vectors inserted into the ANN (HNSW) index.	Shelve each piece next to its similar neighbours.	B4 → C1
11	Quality checks: counts, sample searches, freshness; then the new data goes live.	Taste-test before serving.	B1–C1

4.2 Workflow 2 — Live Recommendation Request

Example: a Group B user types **"comfortable running shoes for flat feet under $100"**.

Step	What happens	Example / simple meaning	Components
1	User submits the search in the app.	The request leaves the phone with user ID and query.	D0 → D1
2	Gateway checks the token and rate limits.	"Are you allowed in?" — yes.	D1
3	Gateway gets the user's A/B group.	This user is in Group B, so use the new system.	D1 → E1
4	Request sent to the Orchestrator.	The manager receives the job.	D1 → D2
5	Orchestrator checks the cache.	Not asked recently — cache miss, continue.	D2 ↔ D9
6	Orchestrator plans and starts agents in parallel.	"Query agent and user agent, go!"	D2
7a	Query agent extracts intent and filters, rewrites the query and embeds it.	Category = running shoes; need = arch support / stability; max price = $100; query vector created.	D3
7b	User context agent loads profile.	Size 9, prefers brand X, browsed trail shoes yesterday.	D4 ↔ C4
8	Retrieval agent runs vector search.	Finds "stability shoe with arch support" reviews even without the words "flat feet".	D5 ↔ C1
9	Retrieval agent runs keyword search.	Catches exact matches like "flat feet" and specific model names.	D5 ↔ C3
10	Results filtered and merged (RRF); ~100 candidates.	Two librarians' piles combined; only in-stock, under $100.	D5
11	Re-ranking agent scores candidates with cross-encoder + user and business signals; adds diversity.	Expert picks the best 15 for this person.	D6 ↔ C2
12	Orchestrator builds the prompt for the LLM.	Rules + user need + profile + 15 products with facts and review snippets.	D2 → D7

Step	What happens	Example / simple meaning	Components
13	LLM selects and orders the best items with reasons.	"Top 5; #1 has firm arch support and great reviews from flat-footed runners."	D7
14	Guardrails validate IDs, price, stock, format, policy.	Supervisor double-checks; all good.	D8 ↔ C2
15	Result cached and returned to the app.	The list appears on screen.	D8 → D9, D0
16	App logs impression; later clicks and purchases.	Tally counter records what the user does.	D0 → E2

Typical time budget for one request

Stage	Typical time	Note
Gateway + A/B lookup + cache check	5–15 ms	Very fast lookups
Query understanding + user context (parallel)	50–150 ms	Small fast model; runs at the same time
Hybrid retrieval (vector + keyword)	20–60 ms	ANN index makes this fast
Re-ranking	50–150 ms	Only on ~100 candidates
LLM generation	200–600 ms	Largest cost; can be streamed or skipped for simple pages
Guardrails + response	10–30 ms	Simple checks
Total	≈ 0.4–1.0 s	Cached answers return in ~10 ms

What if something fails?

Fallbacks keep the page working. If the LLM is slow or down, the re-ranked list is shown without written reasons. If retrieval returns too little, the query is relaxed or popular items in the category are shown. If the whole new pipeline fails, the user is served by the old system. Every fallback is logged so the team can fix it.

4.3 Workflow 3 — Measuring the 35% (A/B Test)

1. **Define success first.** Before starting, the team writes down the primary metric (e.g. hit rate: share of recommendation lists where the user clicked or bought at least one item) and guardrail metrics (latency, errors, revenue).
2. **Estimate sample size.** Calculate how many users are needed to detect the expected change with confidence (power analysis).
3. **Randomly split users.** E1 assigns each user to A (old) or B (new), stable over the whole test.
4. **Serve both systems in parallel.** Same time period, same kinds of users — only the recommender differs.
5. **Record everything.** E2 logs impressions, clicks, purchases and timings, each tagged with the group.
6. **Compute metrics.** E3 computes daily metrics per group.
7. **Compare and test.** E4 calculates lift and runs a significance test.
8. **Check guardrails.** Confirm latency, error rate and cost stayed acceptable.
9. **Decide.** With a significant, healthy win, roll the new system out to everyone.

Worked example (illustrative numbers)

	Group A (old)	Group B (new)
Users in test	500,000	500,000
Recommendation lists shown	4,000,000	4,000,000
Lists with a relevant click/purchase	800,000	1,080,000
Hit rate (accuracy)	20.0%	27.0%
Relative lift	—	(27.0 − 20.0) ÷ 20.0 = +35%
p-value	—	< 0.001 (very unlikely to be luck)
Median latency	180 ms	620 ms (within the 1 s budget)

4.4 Workflow 4 — Feedback Loop (getting better over time)

1. Clicks, purchases and skips from E2/E3 flow back into the activity logs (A3).
2. They become training labels: “for this query, users chose this item”.
3. The embedding model (B4) and re-ranker (D6) are periodically fine-tuned on these labels.
4. Prompts for the LLM (D7) are improved using cases where users ignored suggestions.

5. Each improvement is tested again with a new A/B test before rollout — so accuracy keeps climbing safely.

5. Architecture Diagram

The diagram below shows every component and every connection. Read it top-to-bottom: data is prepared at the top (A → B → C), live requests are served in the middle (D), and results are measured at the bottom (E).

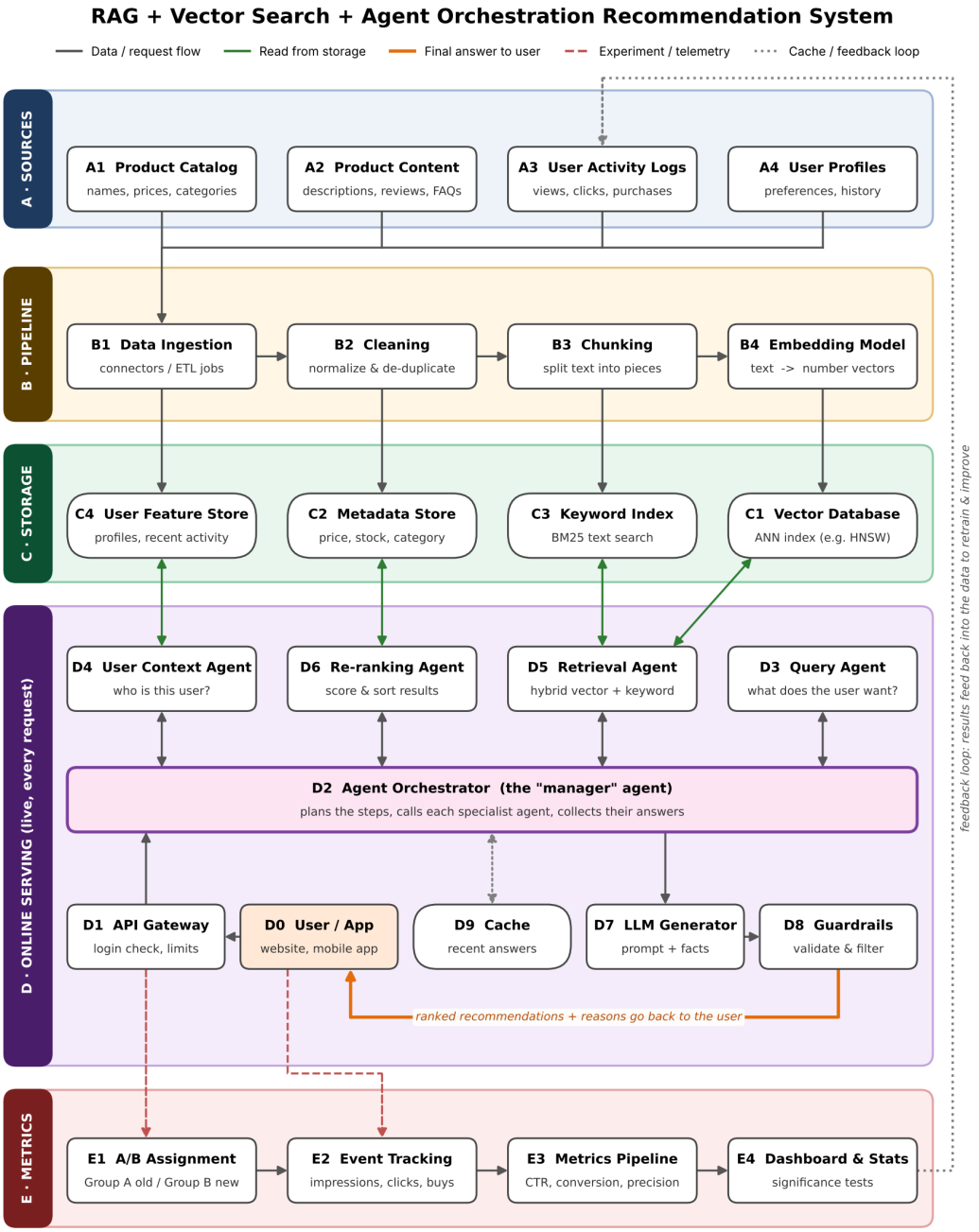


Figure 1. End-to-end architecture of the RAG + vector search + agent orchestration recommender

5.1 How to read the diagram

Visual element	Meaning
Coloured bands with a side label (A–E)	The five layers: Sources (blue), Pipeline (yellow), Storage (green), Online Serving (purple), Metrics (red).
Square-cornered boxes	Processing components that do work (programs, models, agents).
Round-cornered boxes	Storage components that hold data (databases, indexes, cache).
Peach box (D0)	The user / app — where requests begin and answers arrive.
Wide pink box (D2)	The Agent Orchestrator, the central coordinator.
Solid grey arrow	Data or request flowing in the arrow's direction.
Double-headed arrow	A request goes one way and an answer comes back.
Green arrow	An agent reading from a storage system.
Thick orange arrow	The final recommendations returning to the user.
Red dashed arrow	Experiment and telemetry information (A/B group, events).
Grey dotted line	Cache lookup, and the long feedback loop from metrics back to the data.

5.2 Every box on the diagram

Box	Layer	Role in one line
A1 Product Catalog	Sources	Structured facts about each product.
A2 Product Content	Sources	Descriptions, reviews and FAQs — the meaning-rich text.
A3 User Activity Logs	Sources	What users view, click and buy; also receives the feedback loop.
A4 User Profiles	Sources	Preferences and history of each user.
B1 Data Ingestion	Pipeline	Collects all sources into one place.
B2 Cleaning	Pipeline	Normalises and de-duplicates data.
B3 Chunking	Pipeline	Splits text into focused pieces.

Box	Layer	Role in one line
B4 Embedding Model	Pipeline	Turns pieces into meaning-vectors.
C4 User Feature Store	Storage	Ready-made user features for personalisation.
C2 Metadata Store	Storage	Price, stock, category facts.
C3 Keyword Index	Storage	Exact-word (BM25) search.
C1 Vector Database	Storage	Meaning-based nearest-neighbour search (HNSW).
D4 User Context Agent	Online	Finds out who the user is.
D6 Re-ranking Agent	Online	Scores and orders the shortlist.
D5 Retrieval Agent	Online	Hybrid vector + keyword search for candidates.
D3 Query Agent	Online	Understands what the user wants.
D2 Agent Orchestrator	Online	Plans and coordinates all agents.
D1 API Gateway	Online	Front door: security, limits, A/B routing.
D0 User / App	Online	Where requests start and results appear.
D9 Cache	Online	Short-term memory of recent answers.
D7 LLM Generator	Online	Writes the grounded final recommendation.
D8 Guardrails	Online	Validates before showing to the user.
E1 A/B Assignment	Metrics	Puts each user in Group A or B.
E2 Event Tracking	Metrics	Records views, clicks, purchases.
E3 Metrics Pipeline	Metrics	Computes accuracy and health numbers.
E4 Dashboard & Stats	Metrics	Shows lift and proves significance.

5.3 Every arrow on the diagram

From → To	What travels along it	Simple meaning
A1, A2, A3, A4 → B1	Raw records (merged on one line)	All raw ingredients go to the collection point.
B1 → B2	Unified raw records	Collected data goes to be cleaned.

From → To	What travels along it	Simple meaning
B2 → B3	Clean text	Clean text goes to be cut into pieces.
B3 → B4	Text chunks	Pieces go to get their meaning-numbers.
B1 → C4	User features	Customer facts saved for quick lookup.
B2 → C2	Structured facts	Prices, stock, categories saved.
B3 → C3	Chunk text	Pieces added to the word index.
B4 → C1	Vectors + metadata	Meaning-numbers shelved in the vector DB.
D0 → D1	User request	The customer asks.
D1 → D2	Verified request + A/B group	Receptionist passes the job to the manager.
D2 ↔ D9	Cache lookup / store	“Did we just answer this?”
D2 ↔ D3, D4, D5, D6	Tasks out, results back	Manager gives each specialist a task and gets answers.
D4 ↔ C4	User ID / user features	Look up the loyalty card.
D6 ↔ C2	Product IDs / price, stock, rating	Check facts while ranking.
D5 ↔ C3	Keywords / keyword matches	Exact-word search.
D5 ↔ C1	Query vector / nearest neighbours	Meaning search.
D2 → D7	Prompt with retrieved facts	Hand the evidence to the writer.
D7 → D8	Draft recommendations	Writer hands the draft to the supervisor.
D8 → D0 (orange)	Approved recommendations + reasons	The answer reaches the customer.
D1 → E1 (red dashed)	User ID / group label	Which experiment group is this user in?
D0 → E2 (red dashed)	Impressions, clicks, purchases	Record what the customer did.
E1 → E2 → E3 → E4	Group labels, events, metrics	Count, calculate, then judge the result.

From → To	What travels along it	Simple meaning
E4 → A3 (grey dotted)	Learned outcomes / labels	Lessons flow back to improve future models.

6. Why Accuracy Improved by 35%

No single trick produced the gain. Each part fixed a specific weakness of the old keyword-and-popularity system:

Old weakness	What fixed it	Effect
Missed items described in different words	Embeddings + vector search (B4, C1, D5)	Finds items by meaning, e.g. “arch support” for “flat feet”.
Vector search alone misses exact model names	Hybrid search with BM25 (C3, D5)	Exact matches are never lost.
Vague or misspelled requests	Query Understanding Agent (D3)	Clean intent and filters lead to better candidates.
Same results for everybody	User Context Agent + Feature Store (D4, C4)	Results fit the individual.
Rough first-pass ordering	Cross-encoder re-ranking (D6)	Best items move to the top positions users actually see.
Could not reason over reviews and needs	RAG with an LLM (D7)	Weighs evidence and explains choices, grounded in real data.
Wrong or unavailable suggestions	Guardrails (D8) + live metadata (C2)	Fewer bad suggestions; more trust and clicks.
One rigid pipeline for every case	Agent orchestration (D2)	Right steps for each request, retries and fallbacks.
Improvements were guesses	A/B telemetry + feedback loop (E1–E4)	Every change is proven, and the system keeps learning.

7. Typical Technology Choices

The original statement does not list tools; these are common, interchangeable choices for each component.

Component	Common options
Ingestion & scheduling (B1)	Apache Airflow, Kafka, Spark, AWS Glue, dbt
Embedding model (B4)	OpenAI text-embedding-3, Cohere Embed, Sentence-Transformers (e.g. all-MiniLM, BGE, E5), fine-tuned variants
Vector database (C1)	Pinecone, Milvus, Weaviate, Qdrant, pgvector, FAISS, Elasticsearch kNN
Keyword index (C3)	Elasticsearch, OpenSearch, Apache Solr
Metadata / feature store (C2, C4)	Redis, PostgreSQL, DynamoDB, Feast
Agent orchestration (D2)	LangGraph, LangChain, LlamaIndex, Semantic Kernel, custom Python services
Re-ranker (D6)	Cross-encoders (e.g. ms-marco MiniLM, BGE-reranker), Cohere Rerank, gradient-boosted ranking models
LLM (D7)	Hosted models such as GPT-4-class, Claude, Gemini, or open models such as Llama / Mistral
API & serving (D1)	FastAPI, Kong / API Gateway, Kubernetes
Cache (D9)	Redis, semantic cache layers
Experimentation & telemetry (E1–E4)	Optimizely, Statsig, GrowthBook, in-house; Kafka → BigQuery / Snowflake; Grafana, Looker, Tableau
Observability	OpenTelemetry, LangSmith, Prometheus, Datadog

8. Explaining It in an Interview

8.1 The 30-second version

> "We replaced a keyword-based recommender with a retrieval-augmented system. Products and queries are embedded into vectors and searched with a hybrid vector-plus-keyword index. A set of agents, coordinated by an orchestrator, understand the query, pull user context, retrieve and re-rank candidates, and then an LLM produces grounded recommendations that pass guardrail checks. We validated it with an A/B test on live traffic, and it improved recommendation accuracy by 35% with latency still under one second."

8.2 Likely follow-up questions

Question	Short answer
How did you define "accuracy"?	A pre-agreed primary metric such as hit rate or precision@10 from real clicks/purchases, compared between A and B.
Why hybrid search, not only vectors?	Vectors capture meaning but can miss exact tokens like model numbers; BM25 catches those. Fusion (RRF) combines both.
How did you stop hallucinations?	Grounding: the LLM may only pick from retrieved IDs; guardrails verify IDs, price and stock; fallback to the ranked list.
How did you keep latency low?	Parallel agents, ANN indexes, re-ranking only the shortlist, caching, and time-outs with fallbacks.
Why agents instead of one pipeline?	Each agent is simpler, testable and replaceable; the orchestrator adapts steps per request and can retry.
How do you know the 35% was not luck?	Random assignment, sufficient sample size, multi-week duration and a significance test ($p < 0.05$ or better).